From messy exports to a coach dashboard

Data Science 1 · Lab 3: Data Processing I

dr. Maarten Slembrouck

You are the club's new data analyst

`athletes.csv`

43

athletes · 7 columns

ID, birthdate, gender, position, height, weight

`tests.csv`

114

test results · 8 columns

10 m / 30 m sprint, jump height, Yo-Yo level, 3 test moments

`hr_session.csv`

12,000

seconds of data · 5 athletes

Heart rate and speed per second during one training

Messy on purpose: *inconsistent labels, sprint times in milliseconds, missing values, a duplicated athlete and comma decimals.*

Three sessions, one storyline



Lab 3

Data Processing I

From file to tidy table

Load · explore · filter · new columns · clean

Lab 4

Data Processing II

Combine, summarize, reshape

groupby · merge · melt / pivot · time series

Lab 5

Data Visualization I

Tell the story with plotly

Chart choice · plotly express · interactivity · dashboard

How the labs work

Reactive notebooks

Change a cell and every cell that depends on it re-runs, in dependency order - not top to bottom.

One name per variable

Two cells cannot define the same variable. Name each step: tests_raw → tests_clean.

Fill in the …

Every exercise has a … placeholder. The hidden cell below it turns green when your answer is right.

Run it anywhere

Locally with marimo, or online at marimo.app: the notebook then reads the data from GitHub.

LAB 3 · DATA PROCESSING I

From file to tidy table



- Load a dataset and get a first impression
- Select and filter rows and columns
- Add derived columns
- Clean text, numbers, missing values and duplicates

Five methods for a first impression

`.head()`	the first few rows
`.shape`	(rows, columns)
`.dtypes`	the type pandas guessed per column
`.info()`	types, memory and non-null counts
`.describe()`	count, mean, spread, min / max

```
import pandas as pd

athletes_raw = pd.read_csv(
    f"{DATA_DIR}/athletes.csv"
)
athletes_raw.head()
athletes_raw.shape    # (43, 7)
athletes_raw.describe(include="all")

# interactive, sortable table
mo.ui.table(athletes_raw, page_size=10)
```

Filtering finds the first problem

```
# columns
athletes_raw[["name", "position"]]

# rows: a boolean condition inside .loc
athletes_raw.loc[
    athletes_raw["position"] == "Forward"
]

born = pd.to_datetime(athletes_raw["birthdate"])
younger_than_16 = born > "2010-09-01"

tests_raw.sort_values("sprint_30m_s",
                      ascending=False)
```

2 of 14

forwards match the exact spelling "Forward"

The others are written "forward", "forward " or "FWD". Filtering on text only works once the labels are cleaned.

New columns are just assignments

```
# text with a comma -> number
weight = (athletes_raw["weight_kg"]
          .str.replace(",", ".")
          .astype(float))

# age on a fixed reference date
REFERENCE_DATE = pd.Timestamp("2026-09-14")
age = (REFERENCE_DATE - born).dt.days // 365

# speed from a sprint time
speed_kmh = 30 / tests_raw["sprint_30m_s"] * 3.6

df["bmi"] = weight / (df["height_cm"] / 100) ** 2
```



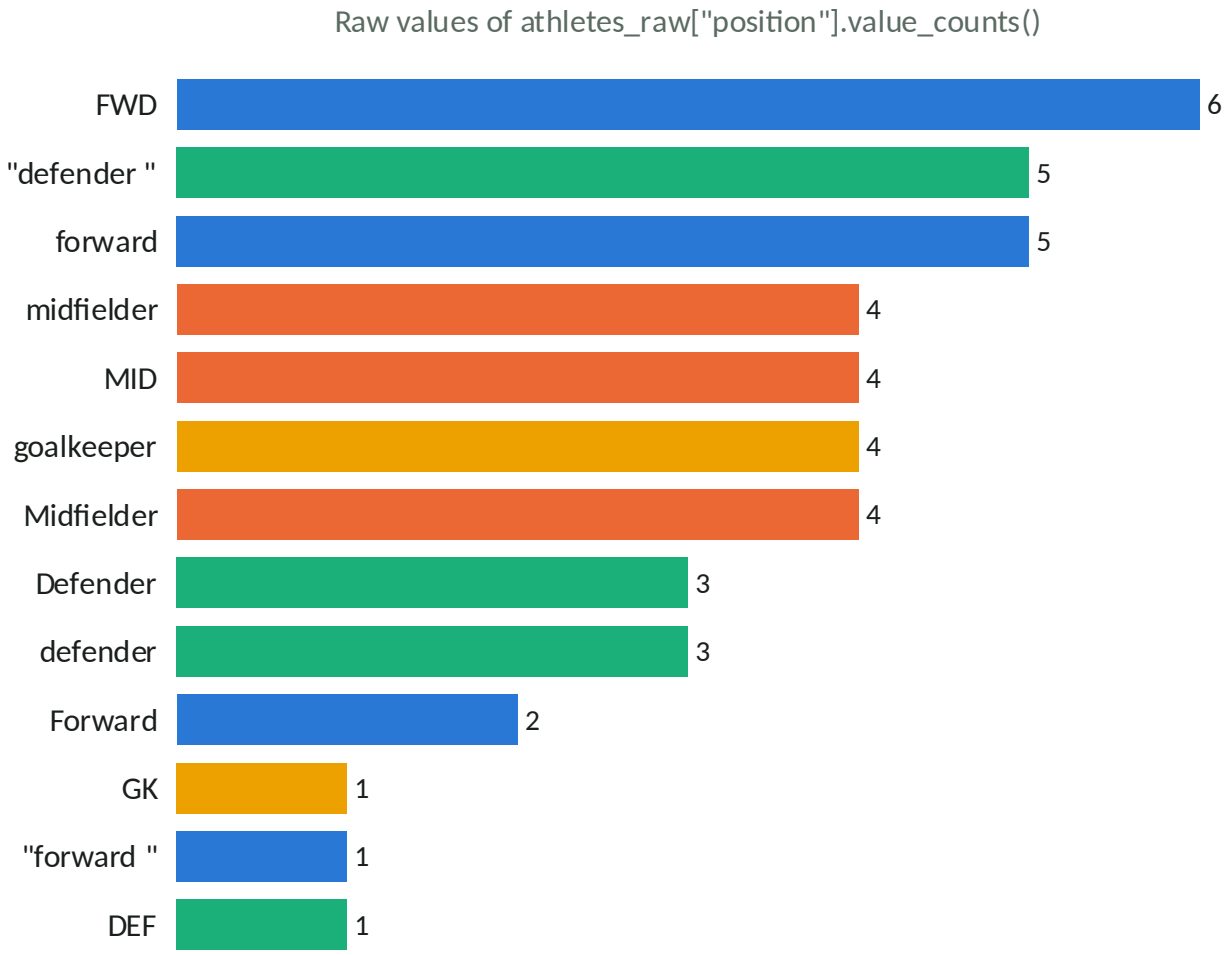
Over 1000 km/h

is the maximum speed this computes. A derived column is often the quickest way to spot impossible values.

`"57,5"`

European-locale export: weight is text until the comma becomes a dot.

13 labels for 4 positions



- Forward
- Midfielder
- Defender
- Goalkeeper

```
(athletes_raw["position"]
    .str.strip()
    .str.lower()
    .map(POSITION_MAP))
# fwd -> Forward, gk -> ...
```

A cleaning toolkit



Missing values

dropna() or fillna()? Decide per column and justify it.

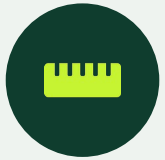
```
isna().sum()
```



Text

Strip spaces, lower the case, then map the abbreviations.

```
.str.strip().str.lower()
```



Units

Above 20 "seconds" is milliseconds: 12 + 17 sprint values.

```
.where(s < 20, s / 1000)
```



Duplicates

Same name and birthdate, two IDs: keep the first row.

```
drop_duplicates(...)
```

Lab 3 exercises

1. **Load tests.csv**
   read_csv and .shape

2. **Forwards under 16**
   combine two conditions

3. **Suspicious sprints**
   filter > 20 and sort

4. **BMI**
   a new column from two others

5. **Age in years**
   dates and timedeltas

6. **Clean positions**
   replace a column

7. **Fix sprint units**
   apply a helper function

8. **Remove the duplicate**
   drop_duplicates

★ **Challenge: top 5 per position**
merge, best sprint, sort, groupby().head(5)